全自動並列AI駆動開発システムのしくみと現在

電DE電LAB. #5 SPECIAL

石川栄和@Vektor,Inc.

Vektor,Inc.
Web Solutions

VK Orchestrator

全自動並列AI駆動開発システムを自作し、
実際の開発に使っています。

処理対象として登録した GitHub issue をオーケストレーターが
自動で拾い、Claude Code が一連の工程を進めます。

仕様検討 / 実装 / レビュー・修正 / 各種テスト / PR作成

複数のタスクを並行処理し、
作業ペインごとに進行状況を表示します。

GitHub issue から PR まで

処理対象のissueを指定すると作業用ペインが自動で開き、
Claude Code が着手します。

STEP	処理
01	**issueを登録**　対象issueへ自分をアサインし、自動処理に指定
02	**実行ちに指定**　オーケストレーターへ取り込み、実行承認にする
03	**ペインが自動で開く**　作業用ターミナルを立ち上げて着手
04	**実装からテストまで**　仕様検討・実装・テスト・修正・PR作成
05	**マージ・片付け**　設定に応じて自動マージ、issue クローズ・環境破棄

今日の内容

前半 7月末までの流れ

ルール、スキル、役割分担を整え、現在のシステムへ至った流れを短く紹介します。

後半 8月末以降の改善

トークンと処理時間、人間の確認負担、
サブ issue の全自動処理を紹介します。

前半部分は

- みなさん既にご存知の方も多い
- 以前セミナーしてるので資料も動画もある

今日はざっと流します。
詳しく知りたい方は過去の資料参照ください。

https://www.vektor-inc.co.jp/post/vws-49-report/

AI が参照するルールの整備

Vektor,Inc.
Web Solutions

コーディング規約などを指定

実装にあたってAIに守ってもらうルールを設定

- 一般的にスタンダードなコーディング規約
  - WordPressなら WordPress Coding Standards
  - Pythonなら PEP 8 (命名、整形、docstring)
  - Goなら gofmt と Go のレビュー規約

その他事前に用意するルール例

- チーム内の共通独自ルールなど
  - 関数やメソッドの命名規則やディレクトリ構造など
    coding-rules.md
  - CSSの命名規則やネストやディレクトリ構造など
    css-rules.md
  - デザインルール（余白のルールとか...）
    design-rules.md
  - PHPUnit テストの形式
    phpunit-rules.md

コーディング以外もルール化

ファイル変更のコミットやプルリクエストの作成を繰り返す

- コミットメッセージを考えるのが面倒
- プルリクの本文を書くのが面倒

→ AI に「コミットして」「プルリク出して」って言えばやってくれるけど...

社内で手動でやってたルールと違うフォーマットだったりする

- コミットメッセージのルール
- プルリクタイトルのルール
- プルリク本文に書く内容や情報の粒度

→ ルール化

余談 : 例えばプルリクのタイトルは changelog / 更新履歴のお知らせページ / SNSでの更新通知 などにそのまま反映させたりするのでとても重要

スキルの作成



スキルとは？

- よく行う作業の手順書のようなもの
- コマンドで簡単に呼び出せる

- ルールのファイルがある上で AI に「プルリク出して」と伝えてもルールファイルを読み込んでくれなかったりする
- 「pull-request-rules.md を読んでルールに沿ってプルリク出して」と毎回言うのもだるい

→ スキル化

スキル例

```
---
name: vk-pr
description: "PR ルールに従い、コミット・changelog 記載・確認手順を含む
---

# /vk-pr スキル

このスキルは `rules/pull-request.md` の PR ルールに従い、**PR 作成まで

～ 以下略  ～
```

/vk-pr と叩くだけで勝手に
ルールに沿ったプルリクを出してくれる

人間がボトルネックになる

開発・PR作成をAIがするようになる
→ レビューの数が増える
→ 手動で人間が確認とかめんどくさくてやってられない

→ レビュー用のスキルを作る

※ 標準で code-review ってスキルもあるけどね...

レビュー用のスキルも作る

ルールがあっても実装者が100%それに従うとは限らない。

- タイトルやプルリク本文がルール通りになってるか？
- 各種コーディングルールが守られているか？
- PHPUnit / e2eテスト がルール通りに書かれているか？

ルールチェックに加えて...

- セキュリティチェック / UXチェック
- e2eテストを実施＆スクリーンショット撮影
- 結果報告

→ スキルとして登録

/vk-pr-review って打てば全部自動レビューしてくれる！

ルールとスキルの棲み分けが効いてくる

- 実装やプルリクを立てる時
- レビューする時

→ ルールは同じ

実装スキルとレビュースキルで同じ注意事項を個別に書くと...
二重、三重管理になっていく。

- ルールのばらつき
- 片方の変更・追加漏れ

実装のスキルもレビューのスキルも、
同じルールファイルを参照させる。

サブエージェントの作成

Vektor,Inc.
Web Solutions

さらに自動処理したい

「プルリク作成スキル」「レビュースキル」ができるなら...

- issue に対する実装前の仕様検討・提案
- 実装後にレビューでひっかかった場合の自動修正

→ 組み合わせたスキルを作れば issue に対して
検討 → 実装 → テスト → レビュー（ 必要に応じて修正 ）
まで一気にやってくれる自動化スキル作れそうよね？

1人のAIに全部やらせると...

全部同じAIが、自分で実装して、自分でレビューすることに...

→ 役割を持ったサブエージェントを作る

- ディレクター
- 実装担当
- UI / UX担当
- ブラウザテスト担当
- コード品質・セキュリティ担当

役割ごとに、参照するルール・使うスキル・完了条件をもった
サブエージェントを定義する

役割を分けるメリット

コンテキストを分離できる

それぞれ必要な専門知識だけを読み込ませられる

並列で作業進められる

特定の機能追加について、実装担当と UI / UX 担当が同時に検討してそれぞれの見解を issue にコメントするなど。

チェックの独立性が上がる

レビュー担当は完成した差分を第三者視点で確認。
「自分が書いたコードだから大丈夫」という前提を持たずに確認できる。

エージェントやモデルを変えられる

- 設計には性能のいいモデル、単純作業にはコストの低いモデルを割り当てたりできる
- 実装エージェントは Claude / レビューエージェントには Codex などを割り振る

作成したエージェント

名前	主な役割
司 （ディレクター）	全体の進行管理・担当者への割り振り
和田（WordPressエンジニア）	主に WordPress 関連などプログラムの実装
植草（UXデザイナー）	UI設計・UX・アクセシビリティ
麗美（レビュワー）	Playwright によるブラウザ動作テスト
安藤（セキュリティ担当）	コード品質・セキュリティの最終レビュー

エージェントの設定をしたら
issue を自動処理するスキルを作ります！

やりたい事

- issue のURLを投げる
- 司さん（ディレクター）が中身を確認
  - 和田くん（開発担当） / 植草君（UX担当） /
    安藤さん（セキュリティー担当）など、
    必要なサブエージェントを起動して仕様検討・提案させる

- 明確な不具合修正や issue で既に仕様がしっかり固まっているものなど、人間の確認が不要
  → 自動で実装開始
- 議論の結果人間に方針確認が必要
  → issue に仕様提案をコメントして指示を待機

- 実装完了したら
  - PHPUnitテスト実行
  - 各種レビュー
    - 安藤さんのセキュリティレビュー
    - 植草君の UX レビュー
    - 麗美ちゃんの e2e テスト
  - 問題があれば差し戻して和田君が修正 → 再レビュー

- 一通り完了したら /vk-pr スキルでプルリク作成
- プルリクの内容がプルリクのルールに沿っているかを司さんが確認
  - 問題があれば修正
- 司さんがプルリクのコメントで人間に最終報告

この一連の処理を一気に実行するスキルを作ります。

issue 自動処理スキル完成！

GitHub の issue の URL を投げれば自動で各種レビュー済みのプルリクまで出してくれる

`/vk-kore {url}` スキル（これやっといてスキル）爆誕！

定期的にチューニングは必要...

スキルができると

「うひょー！これで超ラクだぜ！ヒャッハー！」

そんな事はない...。

意図せず途中で止まったり精度に問題があったりするので、
都度ブラッシュアップの繰り返していきます。

ちゃんと自動処理してもらうために

issue に完了条件を書いておくのは重要

脳内で「これ対応してもらうはずだったのに」と思っても
AIにとっては「そんなん知らんがな。書いとけや。」となる。

無駄な差し戻しや事前につぶしておきましょう。

並列実行



個別の自動処理はできるようになったが...

待ち時間が長い。

- 実装待ち
- テスト実行待ち
- 外部レビュー待ち
- 修正・修正後の再確認待ち

1件が終わるまで待ってから次へ進むのはもったいない。

複数のタスクを同時に動かす

VS Code や Cursol などのエディタで実行するのではなく、
ターミナルを複数開いて同時に処理させる

```
タスク A — 実装 — テスト — レビュー — PR
タスク B — 実装 — テスト — レビュー — PR
タスク C — 実装 — テスト — レビュー — PR
```

標準ターミナルの場合

並列処理自体はできるが数が増えると...

- どのターミナルで、どの issue 処理してるかわからない
- どれが実行中で、どれが入力待ちかわからない
- 終了したセッションを見落とす
- ターミナルを並べ直すだけでも面倒

AIの処理速度より人間の監視がボトルネックに

並列数を増やすほど、
人間がターミナルを見回る仕事が増えていく。

→ 複数セッションの状態を一覧できるUIが必要。

統合制御システム

VK Orchestrator

Vektor,Inc.
Web Solutions

並列実行を円滑にするために

- 進行中 や 実行待ちの issue を見える化
- 人間の入力が必要な状態の見える化
- 後で処理したい issue などをタスクキューとして登録
  - 他の作業の完了後に実行したい
  - トークンリミットの都合で今走らせられない

作った VK Orchestrator の構成

- 作業を実行する AI のルールやスキルなど各種設定
  → VK Agents
- 作業を行う実行GUI
  → VK Terminals
- 進行を制御するシステム
  → VK Orchestrator

処理イメージ

- 自動処理させる issue をタスクキューとして登録
- そのタスクキューをオーケストレーターが定期巡回
- 実行待ち issue 情報を取得して VK Terminals の api に投げる
  - issue のタイトル
  - issue のURL
  - Claude に投げるスキル（ vk-kore ） など

- VK Terminals が受け取って自動的に処理用のペインを開く
- VK Terminals のペイン内の Claude へ自動投入、
  自動処理スキル（ vk-kore ）で プルリクまで作成
  - 入力待ちやマージ待ちなど状態表示をする

エージェントとオーケストレーターの責務

オーケストレーター

- タスクを選ぶ
- 実行場所を割り当てる
- 状態を監視・状態ラベル表示
- 完了条件を判定する
- 次のタスクを流す

エージェント

- 仕様を理解する
- 実装・テスト・レビューを行う
- PRを作成する
- 結果を報告する

オーケストレーターで出来るようになった事

- タスクキューの見える化
- 処理中のペイン（ターミナル）で
  - 処理してる issue タイトルがわかる
  - 処理してる issue のページを簡単に開ける
  - プルリクの URL を簡単に開ける

- 入力待ち状態かどうか
- 人間の確認が不要そうなものは自動マージ
- タスクキューの issue を順番に自動実行

など

最初から全自動を作ったわけではない

1. 不満をルールへ変える
2. 繰り返す作業をスキルへ変える
3. 役割をエージェントへ分けて工程の多いタスクを自動処理できるようにする
4. 複数タスクを並列で動かす
5. 見づらいので統合UIを作る
6. 人間の監視・投入もオーケストレーターへ移す

困りごとを1段ずつ仕組みに変えた結果。

直近２ヶ月での改善事項

トークン消費対策

実装時間対策

人間の確認負荷軽減

複雑な issue の自動処理

Vektor,Inc.
Web Solutions

1. トークン消費対策

実装の実行エンジンの切り替えを可能に

Claude の週間消費はカツカツでもう回せないけど Codex は全然余裕がある場合がある →

実装のサブエージェントだけ Codex で処理する切り替え可能に

◯ トークンの状況を見て消費を分散できるようになった

各工程ごとに実行エンジンを変える

実装だけでなく各工程毎に実行エンジンを切り替えられるように

工程	割り当て例
実装	Codex
UX・UIレビュー	Claude Code
コード・セキュリティレビュー	Codex
E2Eテスト	Claude Code

利用枠の残り方に応じて工程単位で仕事を振り分ける

各工程ごとに実行モデルを変える

実行エンジンだけでなく各工程毎に使用モデルを指定可能に

工程	割り当て例
実装	Codex / Sol
UX・UIレビュー	Claude Code / Sonnet
コード・セキュリティレビュー	Codex / Astra
E2Eテスト	Claude Code / Sonnet

単純な作業はコストの低いモデルを指定する

issue 単位で仕様検討のモデルを変える

実装前の仕様検討は一番重要。
複雑な issue は高性能なモデルを使いたい。

issue	指定	Claude Code
大きい、複雑	`high`	Fable
通常	`mid`	Opus
軽い、実装方法が明確	`low`	Sonnet

実装・レビュー全体を重いモデルへ変えず検討工程だけ指定
※ Codexで仕様検討する場合は別の対応表を使います。

途中で会話を切り替えて続きから再開

メインエージェントが長時間の作業と監視を抱えると、
コンテキストが膨らみ続けます。

1. 工程の区切りで引き継ぎファイルを作る
2. 会話を整理し、新しいセッションへ切り替える
3. 現在のブランチと未コミット差分を確認する
4. テスト結果と残りの工程を引き継いで再開する

最初からやり直さず、コンテキストだけを軽くします。

2. 実装時間対策

issue ごとにレビューを選ぶ

すべての issue にすべてのレビューを実行すると時間もトークンも消費してしまう。

見た目の変更のない作業では UI・UX や e2e をスキップするなど、スキル側で指定はしてあるが、issue 単位でスキップ指定できるように

issueの例	実行するレビュー
文言、文字サイズ、色の変更など軽微な変更	重いレビューはスキップ
条件分岐、権限、複雑な値の変更	UX、セキュリティ、/code-review、CodeRabbitなどを指定

issue 毎に実行・スキップを選びます。

3. 人間の確認負荷軽減

__人人人人人人人人人人人人人人人人人人__
＞　AIの書く文章がわかりにくすぎる　＜
￣Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^￣

issue への実装案、確認事項、PRでの確認事項など

判断に必要な情報を先頭へ

確認文が長いとストレス半端ない。
人間が「何を決めればよいか」を探すことになります。

特に人間判断が必要な issue や PR へのコメントの際は、
何を判断すればいいかわかりやすくなるようにルールを追加

```
[  前半に人間が最低限読むべき内容  ]

＊ 対応概要
＊ 確認してほしいこと
＊ 選択肢と推奨案

――――――――――――――――――――

[  後半は技術情報など実装担当への引き継ぎ用  ]

＊ 技術仕様
＊ 影響するファイル
＊ テスト計画
```

前半だけ読めば返事ができ、後半は実装担当が仕様として読む。

返信テンプレートを用意する

確認項目が多いケースで、長ったらしい確認項目を読んで、手動で順番に回答するのが大変

→ そのままコピペして返せる回答欄を付けるように

```
1. ○○○の表示方法
   回答: A / B / 推奨

2. ○○○の初期状態
   回答: A / B / 推奨

3. 操作後の表示
   回答: A / B / 推奨
```

実際には「すべて推奨で」の一言で返信する事も多いが、確認事項が一覧しやすく、項目が置い場合も回答しやすい。

UI変更が発生する issue は画像で事前確認

UIに関わる仕様提案では変更案のスクリーンショットを issue へ添付するようにルール追加

文章だけの提案	画像を付けた提案
完成形を頭の中で組み立てる必要がある	見た目をその場で比較できる
用語が分からないと判断できない	用語を知らなくても差が分かる
実装後に想定との差が見つかる	実装前にズレを修正できる

4. 複雑な issue の自動処理

サブ issue 分割後は人間が流していた

重い issue は複数のサブ issue へ自動分割はしていたが、
分割しても、その後の進行管理が手作業で残っていました。

分割後に必要だった作業	人間の負担
未決の仕様を確認	各issueを順番に読む
依存関係とマージ順を判断	着手できるissueを選ぶ
自動処理へ投入	1件ずつ実行待ちにする
進行を監視	止まったペインへ声をかける

サブ issue も最後までAIが自動処理

1. 全サブ issue を読み、未決の仕様を先に人間に確認する
2. 依存関係とマージ順を整理する
3. 着手できるissueを自動マージON・実行待ちへ切り替える
4. 並行できる別リポジトリの issue を続けて投入する
5. 作業ペインを監視し、止まったり必要なら声をかける
6. 全サブ issue が完了まで見守る / 親 issue は自動クローズ

同じリポジトリでは衝突を避けて1本ずつ、
別リポジトリを跨ぐ場合は並行して進めます。

自動化後のボトルネックも仕組みにする

7月末までに自動化したこと	その後に軽くしたこと
実装、レビュー、テスト	エンジンとモデルの使い分け
issueからPRまでの処理	人間が読む確認文とUI確認
複数タスクの並列実行	サブ issue の依存管理と完了監視

全自動を一度で作ったわけではありません。

運用して面倒だった箇所を、順番に仕組みへ変え続けています。

ありがとうございました

Vektor,Inc.
Web Solutions